Rust + Algorithms Illuminated Part 1: 16-Week Study Plan

Oct 9, 2026 · @JC

Overview

This plan pairs *The Rust Programming Language* (2023 edition, 20 chapters, free online) with *Algorithms Illuminated, Part 1* (12 chapters, pseudocode only) over 16 weeks. Every algorithm in the second book becomes a Rust exercise.

Assumptions: about 6-8 hours a week, no prior Rust experience, and some programming background. If either is off, stretch or compress the phases.

Weekly rhythm:

1. Read the Rust chapters (2-3 hours).
2. Read the matching algorithms chapter (2 hours).
3. Implement the algorithm in Rust, with tests (2-3 hours).

Phase 1: Rust basics with simple algorithms (Weeks 1-4)

Ownership (Rust book Ch 4) is the hardest hurdle, so spend extra time there. Merge sort is a good test of it because you must decide when to copy and when to borrow slices.

Week	Rust book	Algorithms Illuminated	Build
1	Ch 1 (Getting Started), Ch 2 (Guessing Game)	Ch 1 (intro, grade-school vs. Karatsuba multiplication)	Install Rust, learn Cargo, create and run a Hello, World! program (cargo new, cargo run), write grade-school multiplication on digit vectors
2	Ch 3 (Common Concepts)	Ch 2 (asymptotic notation)	Brute-force and loop-based exercises, e.g. timing O(n) vs. O(n²) functions

Week	Rust book	Algorithms Illuminated	Build
3	Ch 4 (Ownership)	Ch 1 again: MergeSort	Merge sort on `Vec<i32>`, working through borrowing and slices
4	Ch 5 (Structs), Ch 6 (Enums/pattern matching)	Ch 3 (divide and conquer: counting inversions)	Counting inversions, Karatsuba with a custom `BigNum` struct

Phase 2: Collections, errors, generics (Weeks 5-8)

By the end of this phase your sorts are generic, tested, and benchmarked against the standard library.

Week	Rust book	Algorithms Illuminated	Build
5	Ch 7 (Modules), Ch 8 (Collections)	Ch 4 (Master Method)	Organize your code into a library crate with modules, add a benchmark harness
6	Ch 9 (Error Handling)	Ch 5 (QuickSort)	QuickSort with random pivots, and file-reading with proper `Result` handling
7	Ch 10 (Generics, Traits, Lifetimes)	Ch 5 again, then Ch 6 (linear-time selection)	Make your sorts generic over `T: Ord`, implement randomized and deterministic selection
8	Ch 11 (Testing)	Review Ch 1-6	Add unit and integration tests to everything so far, and compare your sorts against `slice::sort`

Phase 3: Graphs (Weeks 9-12)

Use index-based graphs (vertices as `usize`) rather than pointer-linked graphs. They fit Rust's ownership rules much better.

Week	Rust book	Algorithms Illuminated	Build
9	Ch 12 (minigrep project)	Ch 7 (graphs, minimum cut)	Graph as adjacency list (`Vec<Vec<usize>>`), then Karger's min cut (use the `rand` crate)
10	Ch 13 (Closures, Iterators)	Ch 8 (BFS, DFS, topological sort)	BFS and DFS, using iterators where natural
11	Ch 8 revisited and Ch 14 (More on Cargo)	Ch 8 (strongly connected components)	Kosaraju's SCC algorithm on a large edge-list file
12	Ch 15 (Smart Pointers)	Ch 9 (Dijkstra)	Dijkstra with a simple array version first, then `BinaryHeap`

Phase 4: Data structures and capstone (Weeks 13-16)

The last week is a capstone: pick a benchmark suite comparing all your algorithms, or a small CLI tool built on your graph and sorting library.

Week	Rust book	Algorithms Illuminated	Build
13	Ch 16 (Concurrency)	Ch 10 (heaps)	Implement your own min-heap, then a heap-based Dijkstra and a median-maintenance exercise
14	Ch 17 (OOP features), Ch 18 (Patterns)	Ch 11 (search trees)	Binary search tree using `Option<Box<Node>>`, the classic Rust linked-structure exercise
15	Ch 19 (Advanced Features)	Ch 12 (hash tables, Bloom filters)	Hash table with chaining, then a Bloom filter. Compare to `HashMap`
16	Ch 20 (Final Project: web server)	Capstone	Benchmark suite for all your algorithms, or a CLI tool using your graph and sorting library

Tips

- **Do the problems.** Each Algorithms Illuminated chapter ends with problems, and the book's companion site (algorithmsilluminated.org) has programming challenges with test data. Those make good Rust exercises.
- **Ask the compiler.** Read Rust's error messages carefully; they're unusually good. Run `cargo clippy` regularly.
- **Rewrite after each Rust chapter.** Once you learn traits or iterators, go back and refactor an earlier algorithm. This is where much of the learning happens.
- **Supplements:** Rustlings (small exercises that follow the book) and Rust by Example work well alongside it. The 2023 book predates some newer features, such as async coverage in later editions, but nothing here depends on that.
- **Appendices:** Read the algorithms book's appendices on proofs and induction as needed, and skim the Rust book's appendices when you want the details.